NESNEYE YÖNELİK PROGRAMLAMA

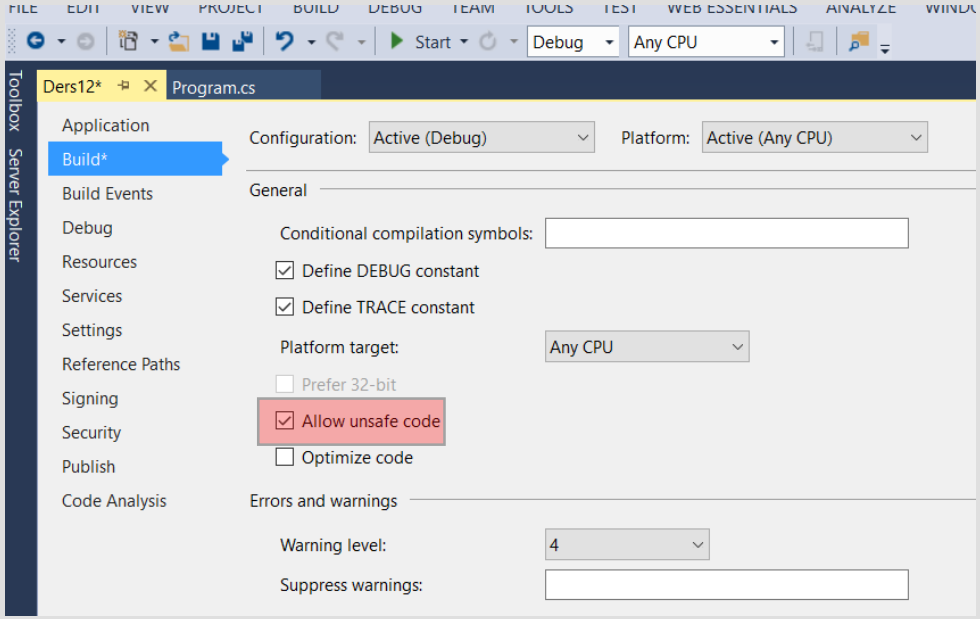
Ders 13: Unsafe Code, Multithreading, UML, Unit Testing, Design Patterns, MVC

Konular

- Unsafe Code
- Multithreading
- UML
- Unit Testing
- Design Patterns
- MVC
- Gelecek Dersler
- Diğer Üniversiteler ve Öğrenim Siteleri…

Unsafe / Unmanaged Code (Güvenli Olmayan / Yönetilemeyen Kod)

- C#, Güvenli olmayan erişim işaretlendiği zaman fonksiyonlarda pointer (işaretci) değişkenlerin kullanımına olanak sağlar.
- Güvenli olmayan kod pointer değişkenleri kulanan bir kod bloğudur.
- Öncelikle Compiler'ın bu kodları derleyebilmesi için Proje üzerine sağ tıklayıp sağ tuş -> Properties -> Build



Pointers

- Bir pointer, adres içindeki değeri veya adresi tutan bir değişkendir.
- Değişken tanımlamada olduğu gibi önce pointer tanımlanır.
- Normal değişkenden farklı olarak pointer tipinin sonuna * karakteri konulur. Örneğin:
  - int* p1, p2, p3; // Tamam
  - int *p1, *p2, *p3; // C#'ta Geçersiz

Pointers

```
class Program
{
    static unsafe void Main(string[] args)
    {
        int var = 20;
        int* p = &var;
        Console.WriteLine("Data is: {0} ", var);
        Console.WriteLine("Address is: {0}", (int)p);
        Console.ReadKey();
    }
}
```

- Yazılan kodların unsafe olduğu metot içinde belirtilebilir. Veya unsafe olan code bloğu olduğu metot içinde de belirtilebilir.
- & işareti ile değişkenin adresi çekilebilir.

Pointers – Veriyi okuma

```
static void Main(string[] args)
{
    unsafe
    {
        int var = 20;
        int* p = &var;
        Console.WriteLine("Data is: {0} ", var);
        Console.WriteLine("Data is: {0} ", p->ToString());
        Console.WriteLine("Address is: {0} ", (int)p);
    }

    Console.ReadKey();
}
```

- Bu örnekte unsafe kısım metot içinde belirtilmiştir.
- Pointer içindeki değeri okumak için p->ToString() metodu kullanılır.

Pointers – Veriyi yer değiştirme

```
class islem {
    public unsafe void swap(int* p, int* q)
    {
        int temp = *p;
        *p = *q;
        *q = temp;
    }
}

class Program
{
    public unsafe static void Main()
    {
        islem _is = new islem();
        int var1 = 10;
        int var2 = 20;
        int* x = &var1;
        int* y = &var2;

        Console.WriteLine("Değiştirmeden önce: var1:{0}, var2: {1}", var1, var2);
        _is.swap(x, y);

        Console.WriteLine("Değiştirdikten sonra: var1:{0}, var2: {1}", var1, var2);
        Console.ReadKey();
    }
}
```

- Bellek bölgesi üzerinden verileri yer değiştirten bir metot yazılmış. Metot ayrı bir sınıftadır.

Aynı işlemi ref ve out konuları ile de yapabileceğinizi unutmayın. Ayrıca ref ve out unsafe code değildir.

Pointers – Array verilerine ulaşma

```
class Program
{
    public unsafe static void Main()
    {
        int[] list = { 10, 100, 200 };
        fixed (int* ptr = list)

            for (int i = 0; i < 3; i++)
            {
                Console.WriteLine("Address of list[{0}]={1}", i, (int)(ptr + i));
                Console.WriteLine("Value of list[{0}]={1}", i, *(ptr + i));
            }

        Console.ReadKey();
    }
}
```

- Bir pointer değişkeni bellekte sabit olmadığı için arttırabilirsiniz. Ama bir dizi bellkete sabittir (fixed) ve arttıramazsınız.
- Bu yüzden, bir pointer değişkeni kullanan bir array verisine erişme durumda (C ce C++ olduğu gibi), bir sabit pointer'a ihtiyaç duyarız. C# fixed anahtar sözcüğü ile bu durum tanımlanır.

Multithreading (Çok kanallı)

- Bir thread bir programın çalışma yolu olarak tanımlanır.
- Her thread benzersiz bir kontrol akışına sahiptir.
- Eğer uygulamanız karmaşık ve zaman alıcı işlemleri içeriyorsa, her thread belirli bir iş parçasını yerine getirmesi için thread'lerin ayarlanması yararlı olur.
- Thread'ler en hafif process'tir. Modern işletim sistemleri thread'leri iyi şekilde yönetebilir. Bu sayede hem CPU iyi kullanılmış olur hem de uygulamaların etkinliği arttırılmış olur.
- Bu güne kadar uygulama geliştirirken genelde tek thread şekilde yazdık. Ancak, uzun bir process olduğunda uygulamamızın sadece o process ile uğraşması çok hantal bir uygulama olmasına sebebiyet verir.
  - Örneğin internetten download işlemi başlatan bir uygulama yazdınız ve diğer download işlemini başlatmak için çalışan download işleminin bitmesini bekliyorsunuz. Sizin bilgisayarınızın hızı Network hızından çok çok hızlıdır. İki, üç ve hatta çok fazla download aynı anda yönetebilir. Bu durumda aynı zamanda birden fazla download'ın nasıl yönetileceği problemi ile karşılaşırız.

Bir thread'in yaşam döngüsü

- C# bir thead'in yaşam döngüsü System.Threading.Thread sınıfından bir nesne yaratıldığında başlar ve thread bitince veya yok edilince biter. Yaşam Döngüsü:
  - Başlamamış Durum: Bir başlama (start) işlemi henüz yapılmamış durumdaki thread.
  - Hazır Durum: Thread başlatılmış çalışma hazır ve CPU'yu bekleme durumunda.
  - Çalışmayan Durum:
    - Thread uykuya alınmış olabilir
    - Thread başka bir thread'i bekleyebilir.
    - Bir I/O işlemi tarafından bloklanmış olabilir.
  - Dead State: Thread çalışmayı tamamladığı veya yok edildiği zaman ki durumdur.

Multithreading

```
using System;
using System.Threading;

namespace Ders12_04_Threads
{
    class Program
    {
        static void Worker()
        {
            for (int i = 0; i < 10; i++)
            {
                Console.WriteLine("İşlem " + i);
                Thread.Sleep(1000); //1. saniye bekle
            }
        }

        static void Main(string[] args)
        {
            Thread myThread = new Thread(Worker);
            myThread.Start();
            Thread.Sleep(1500); //1.5 saniye bekle
            Console.WriteLine("Benim işim bitti...");
            //Bu satıra ulaşmanıza rağmen thread çalışmaya devam eder.
            Console.ReadKey();
        }
    }
}
```

```
İşlem 0
İşlem 1
Benim işim bitti...
İşlem 2
İşlem 3
İşlem 4
İşlem 5
İşlem 6
İşlem 7
İşlem 8
İşlem 9
```

- Öncelikle System.Threading using bölümüne ekleyelim.
- Worker adında metotu thread olarak tanımlayıp start metotu ile thread'i çalıştırmaya başladık.
- Kod bir alt satıra geçmesine rağmen thread arka planda çalışmaya devam etti.

Multithreading - Lock

- Thread'lerin en kötü tarafı ortak değişkenlerin yönetiminde çıkabilir. Eğer uygun bir şart sağlanmadan değişkenin değerini değiştirirsek bu diğer thread'leri de etkiler. Ortak değişken üzerine işlem yaptığımızda veri tutarlığı sağlamak için lock komutu kullanılmaktadır.
- Lock işlemi ile kodunuzun bir kısmını kilitlersiniz. Bu kilit işlemi için ise referans tipi bir nesne kullanmanız gerekmektedir.
- Lock komutu bu nesneyi kilitleyerek erişimi engellemektedir. Aynı nesneyi aynı anda diğer bir iş parçacığı kilitleyemeyeceği için kilitlediğiniz kodlar güvenle çalışabilmektedirler.

```
lock ( kilitlenecekNesne ) {
// kodlarınızı buraya güvenle yazabilirsiniz.
}
```

Multithreading - Lock

```
static class NormalSınıf
{
    static int x;
    static void Arttir() { x++; }
    static void Yukle()  { x = 123; }
}
```

```
static class GuvenliSınıf
{
    static readonly object kilit = new object();
    static int x;

    static void Arttir() { lock (kilit) x++; }
    static void Yukle()  { lock (kilit) x = 123; }
}
```

- Normal sınıfta herhangi bir koruma yok. Aynı nesneyi bir veya birden fazla thread kullanabilir.
- Lock ile aynı anda iki veya daha fazla thread'in kullanması engellenmiş.

Multithreading - Sleep

- Sleep: Thread'in belli bir süre bekletmek için uygulanır.
- Kullanımı milisaniye cinsindendir.

```
Thread.Sleep( 1000 );
```

1000 milisaniye yani 1 saniye bekle.

Multithreading - Abort

- Abort: Thread’i durdurmak için kullanılır.
- İş parçacığınızın Abort metodunu çağırdığınızda zorla kapatılacaktır. Herhangi bir parametre almayan bu metodu çalıştırdığınızda iş parçacığınızın durumu öncelikle AbortRequested olarak değişecektir.
- Bir süre sonra da iş parçacığı durdurularak durumu (ThreadState) Aborted olarak değiştirilecektir.
- Abort metodunu kullanarak bir iş parçacığını durdurduğunuzda ThreadAbortException hatası ile karşılaşırsınız. Bu nedenle bu exception'ı mutlaka kontrol altında tutmanız gerekmektedir. Bunun için ise yapmanız gereken bildiğiniz üzere try…catch kullanmaktır.

Multithreading - Abort

```
static void Worker()
{
    try
    {
        while (true)
            try
            {
                // Kodlarınızı buraya yazabilirsiniz...
                Thread.Sleep(0);
            }
            catch (ThreadAbortException)
            {
                Console.WriteLine("Thread zorla kapatılmıştır...(2)");
                break;
            }
            catch (Exception ex)
            {
                Console.WriteLine("Beklenmeyen Hata: " + ex.Message);
            }
    }
    catch (ThreadAbortException)
    {
        Console.WriteLine("Thread zorla kapatılmıştır...(1)");
    }
    // Burada son işlemlerinizi gerçekleştirebilirsiniz.
}
```

```
static void Main(string[] args)
{
    Thread myThread = new Thread(Worker);
    myThread.Start();
    Thread.Sleep(2000);
    Console.WriteLine(myThread.ThreadState);

    myThread.Abort();
    Console.WriteLine(myThread.ThreadState);

    myThread.Join();
    Console.WriteLine(myThread.ThreadState);

    Console.ReadKey();
}
```

- İç içe iki tane thread kullanılmıştır. 2. hata abort işlemi sonrası 1. tip hata ise tekrar thread kullanılmak istediğinde ortaya çıkar.
- Her durdurma bir ThreadAbortException üretir. Bu durum dışanda bir durdurma olduğu ise o da ikinci catch bloğu ile yakalanabilir.
- İkinci cacth bloğu Exception türündendir ve tüm hataları yakalayabilir.

Multithreading - Abort

- Durağan (static) sınıf constructor'ları kesinlikle abort metodu ile sonlandırılamazlar.
- Catch/finally blokları içinde iken abort metodu çalışmayacaktır.
- Bir diğer durum ise unused bir kod çalıştırmışsanız karşınıza çıkacaktır. Bu durumda abort devreye giremeyecektir.

Multithreading - Interrupt

- Bu metod ile uyku modunda olan bir iş parçacığınızı zorla uyandırabilirsiniz. Bu aslında doğru bir çözüm olmasa da iş parçacıkları arası iletişimde kullanabileceğiniz en basit yoldur. Bu metod ile bir iş parçacığınız uyandırmanız durumunda ThreadInterruptException hata mesajı üretecektir.
- Bu metodun Abort metodundan farkı ise iş parçacığınızı sonlandırmamasıdır. Abort metodu az evvelde bahsettiğimiz üzere bir iş parçacığınız zorla durdurmak için kullanılabilirken Interrupt metodunu sadece uyumakta olan iş parçacığını uyandırmak için kullanabilirsiniz.
- Interrupt metodunu kullanmadan önce iş parçacığınızın uyku modunda olup olmadığını ThreadState özniteliğini kontrol ederek belirleyebilirsiniz.
- Unutmayın ki eğer bir iş parçacığı Interrupt metodu ile çalışmak üzere tasarlanmamışsa bu metodu kullanmak kesinlikle çok tehlikelidir. O nedenle sadece kendi hazırladığınız ve Interrupt metodu ile çalışmak üzere çok dikkatlice tasarladığınız bir iş parçacığında kullanabilirsiniz.

```
static void Main()
{
    Thread isim = new Thread (isciMetod);
    t.Start();
    Thread.Sleep(5000);
    // ön hazırlıkların yapıldığınız varsayalım.
    t.Interrupt();
}

private void isciMetod()
{
    try {
        Thread.Sleep (Timeout.Infinite);
    }
    catch (ThreadInterruptedException) {
        Console.Write ("Zorla Uyandırıldı");
    }
    // Burada işlerimizi yapabiliriz.
}
```

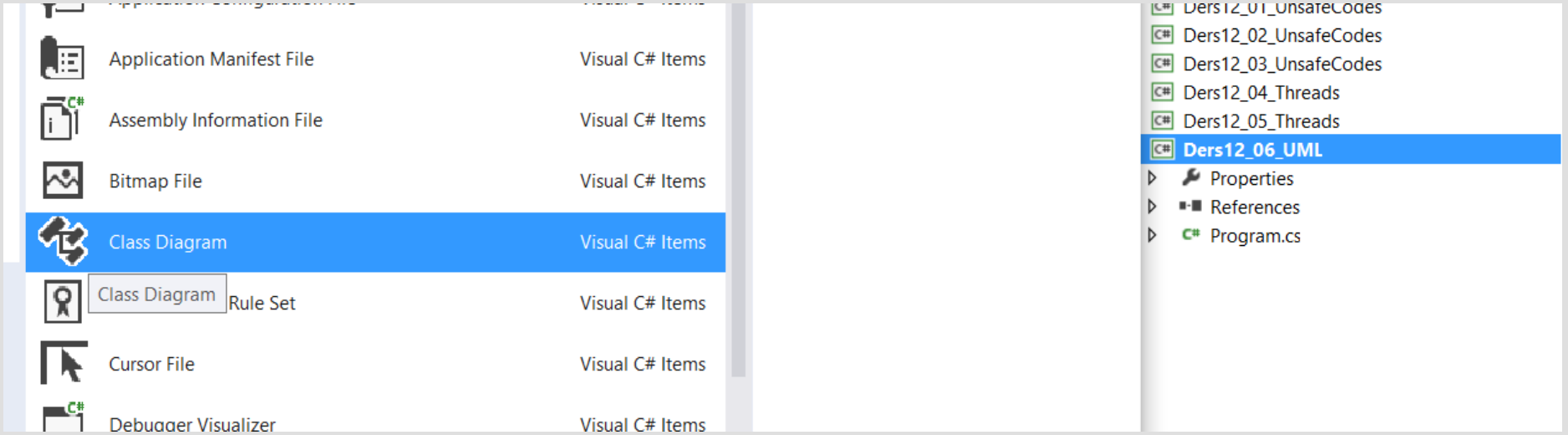
Daha ileri konular

- Bu konuya devam etmeyi düşünürseniz:
  - Senkronizasyon
  - Paralel Programlama
  - Concurrent Collections
  - Asenkron Programlama
- Bu konularda İngilizce bir çok kaynak bulabilirsiniz. Ayrıca aşağıdaki Linki Türkçe olarak hazırlanmış güzel bir doküman:
  - http://ileriprogramlama.blogspot.com.tr/2012/09/multithread-programlamann-temelleri.html

Class Diagramları ve UML

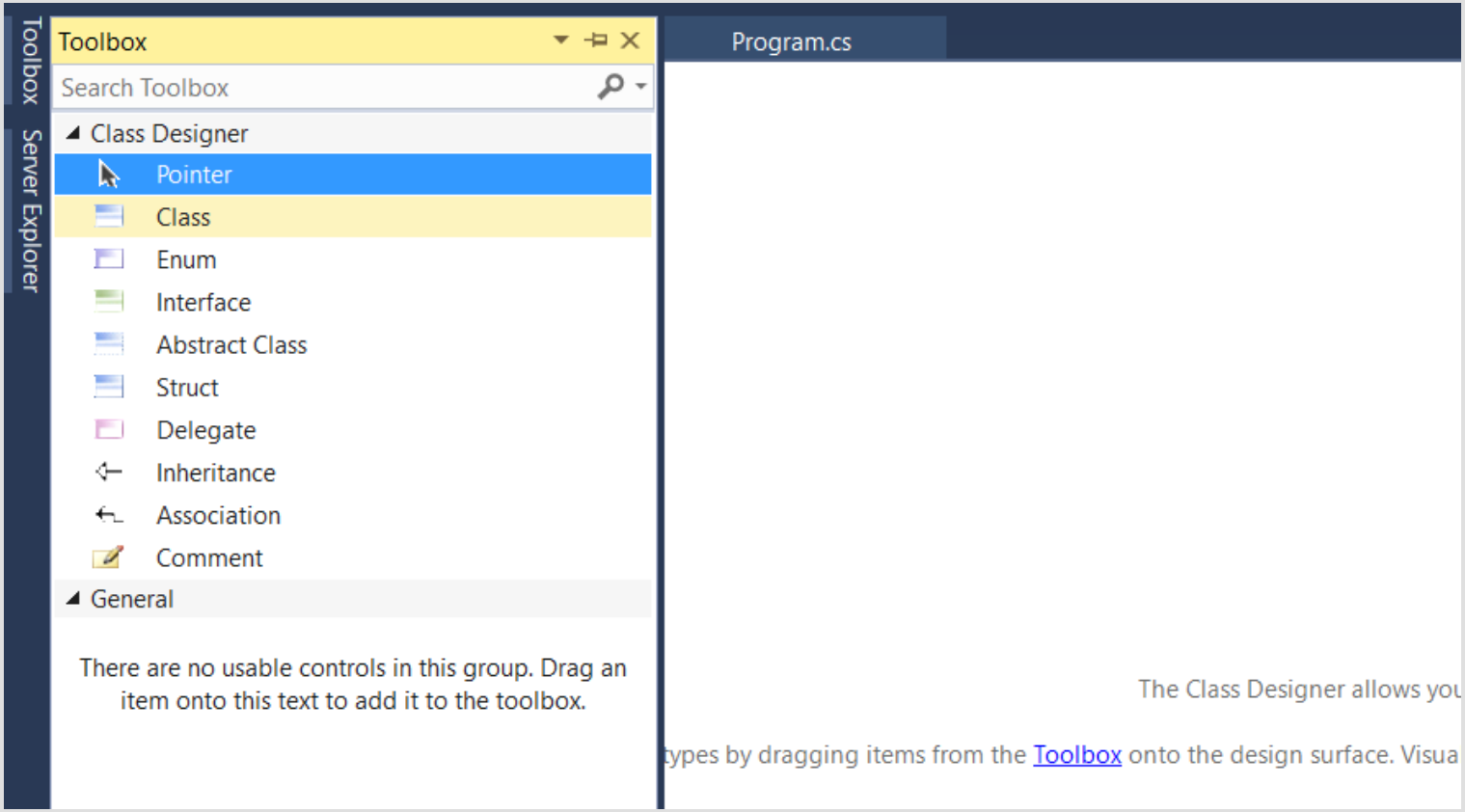
- UML (Unified Modelling Language - tümleşik modelleme dili) grafiksel bir dil olarak tasarlanmıştır.
- UML ile sınıf diyagramları oluşturmak nesne temelli programlamanın getirdiği bir gerekliliktir.
- UML ile oluşturulan "Sınıf Diagramları", aralarında bizim belirlediğimiz ilişkileri içeren sınıflar topluluğudur.

Class Diagramları ve UML



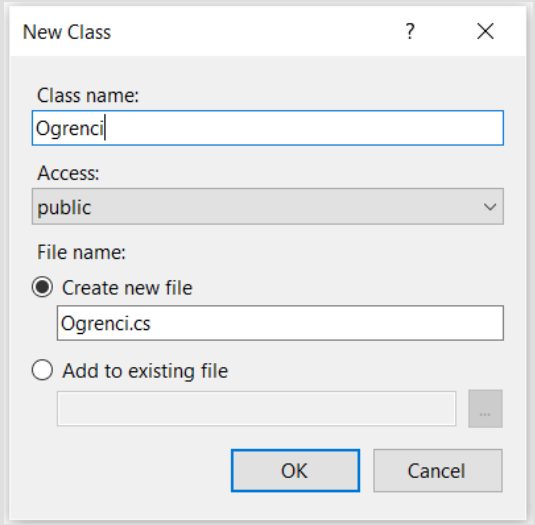
- Projemize ekleme (Sağ Tuş – Add – New Item) yapıp Class Diyagram konusunu seçiyoruz.

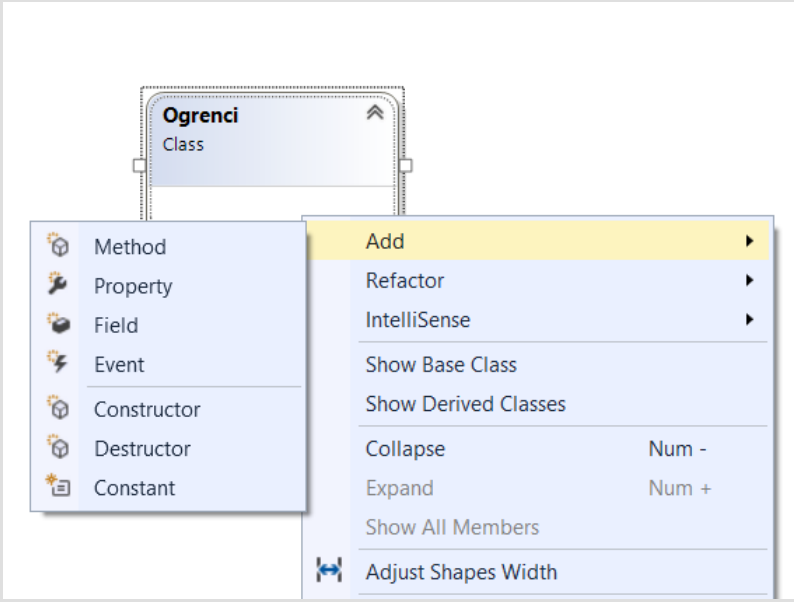
Class Diagramları ve UML



- ToolBox bölümünden açılan konuları biliyorsunuz. NYP dersinde öğrendiğimiz konular ☺
- Sadece Association konusu bilmiyoruz. Bu konu veritabanı dersinde ilişki kurma konusunda anlatılacaktır.

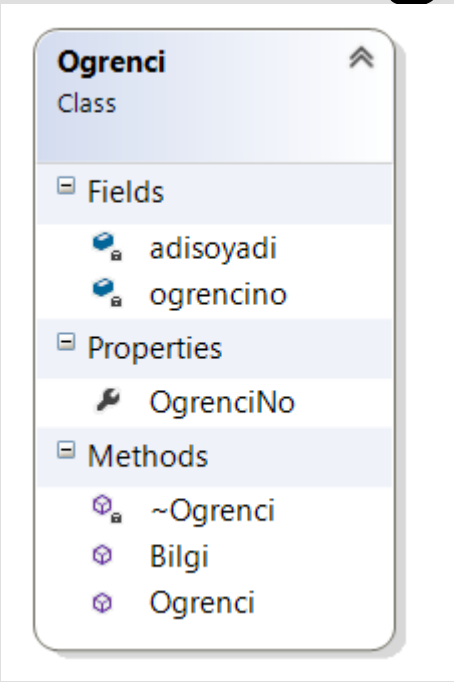
Class Diagramları ve UML

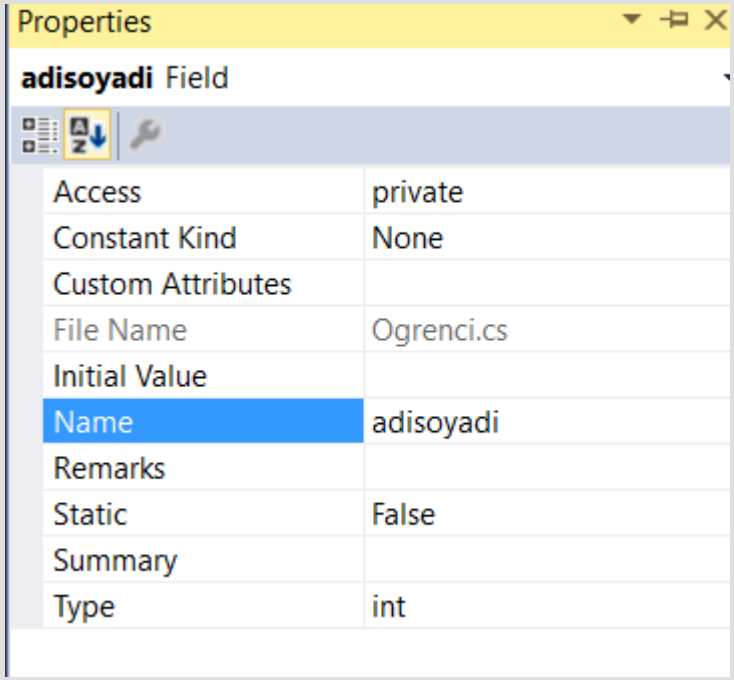




- Ogrenci adında bir sınıf oluşturdum.
- Sınıf üzerinde Sağ tuşa tıklayıp Add dediğimizde gelen konularda tanıdık sanırım ☺

Class Diagramları ve UML





- İki Field, bir property, bir metot ve constructor / destructor ekledim.
- Şimdi seçim yapıp sağ tuş -> Property deyince yine tanıdık konular.
  - Adisoyadi type özelliğini string yapıyorum…
- Görsel sınıfın üzerine tıkladığınızda kod sayfası açılır.

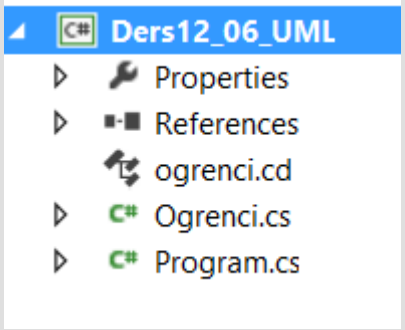
Class Diagramları ve UML

```
using System.Text;

namespace Ders12_06_UML
{
    public class Ogrenci
    {
        private string adisoyadi;
        private int ogrencino;
        public Ogrenci()
        {
            throw new System.NotImplementedException();
        }

        ~Ogrenci()
        {
            throw new System.NotImplementedException();
        }

        public int OgrenciNo
        {
            get
            {
                throw new System.NotImplementedException();
            }
            set
            {
            }
        }
        public void Bilgi()
        {
            throw new System.NotImplementedException();
        }
    }
}
```



- Kod bölümü derste gördüğümüz konular aslında.
- Throw bölümleri şu anlık gata verdiriyor, bu bölümleri görsel kodlama şansımız yok. Oturup yine kod yazmalıyız.
- Solution Explorer bölümnde ogrenci.cd dosyası UML tasarım için iken Ogrenci.cs oluşturduğumuz Ogrenci sınıfı için oluşturulmuş dosyalardır.
- UML projeleri görsel olarak tartışmak ve şablonu geliştirmek için iyidir.
- NYP iyi şekilde bildiğinizde UML'de hızlı şekilde öğrenebilirsiniz.

Unit Testing (Birim Testi)

- Yazılım Mühendisliği dersinde göreceğiniz Yazılım Testi Yöntemlerinden biri de Birim Test'tir.
- Birim test, programımızın içindeki class ve modüllere yönelik yazılmıştır.
- Test edilecek kodun sadece Input/Output kontrolünü yapar. Her döngü ve değişkenlerini veya programlama mantığınızı test etmez.
- Test aşaması programın en uzun süren ve test sonunda yapılan düzeltmelerin yeni hatalara sebebiyet verebildiği aşamasıdır.
- Bu noktada, derleme aşamasında yapılan bu test için C# (Nunit) ve Java (JUnit) gibi dillerin Unit Test desteği vardır.
- Daha geniş bilgi için «Test Driven Development ve Unit Testing» anahtar kelimelerini kullanabilirsiniz.

Unit Testing

```
using System.Text;

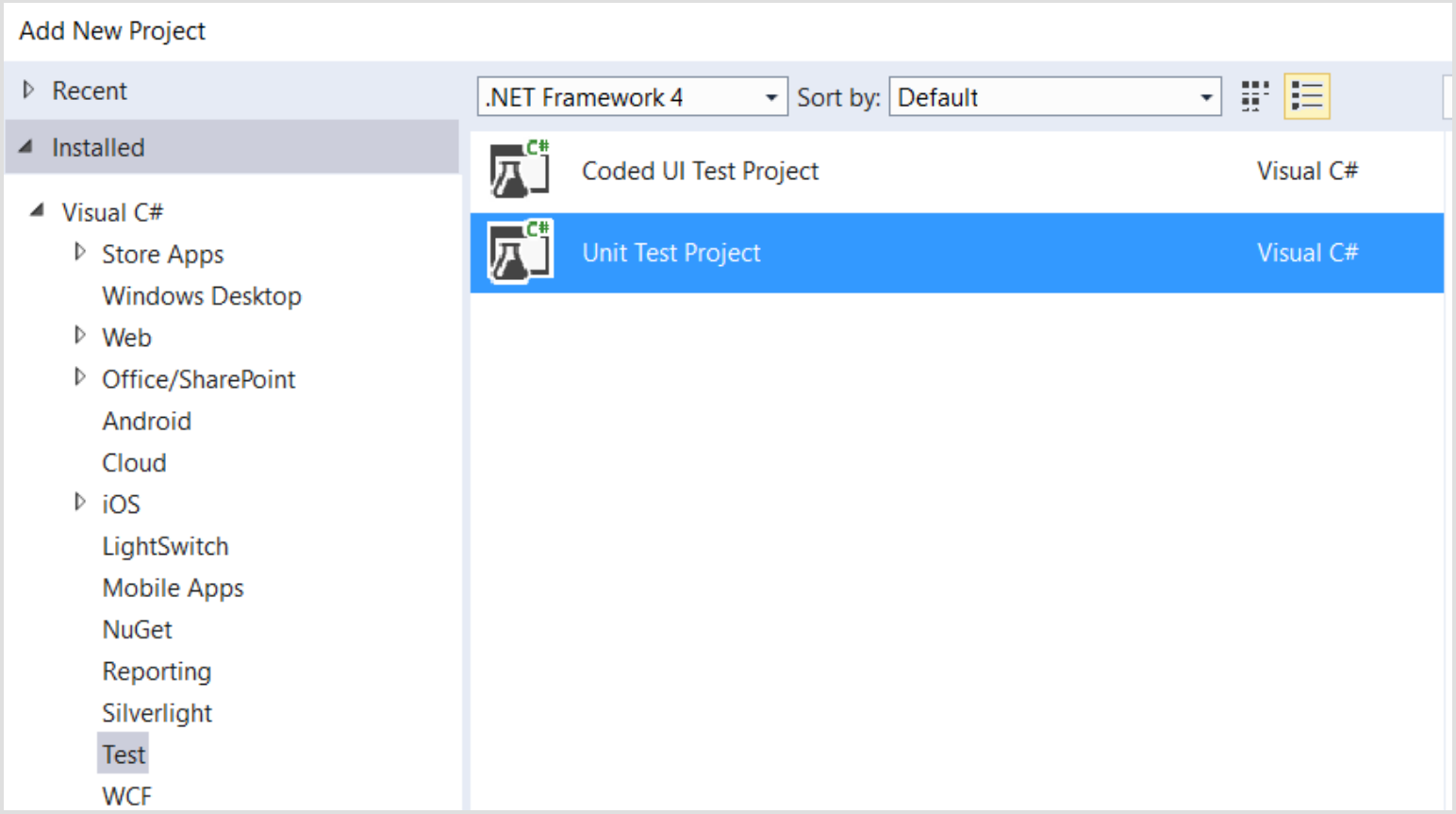
namespace Ders12_07_SimpleLibrary
{
    public class SimpleClass
    {
        //Array'i birer boşluklu birleştiren ve sonucu bir string içine atıp o string'i döndüren kod
        public string combineArrayStringWithSpace(string[] stringArray)
        {
            string str = "";

            foreach (string item in stringArray)
            {
                str += item + " ";
            }

            return str.Trim();
        }
    }
}
```

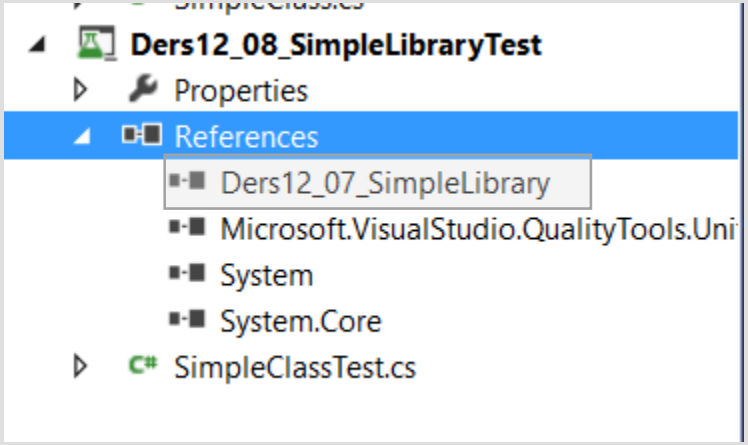
- Öncelik bir metottan oluşan basit bir sınıf tanımlayalım.
- Metot bir string array alsın ve dizinin elemanları arasına bir boşluk bıra

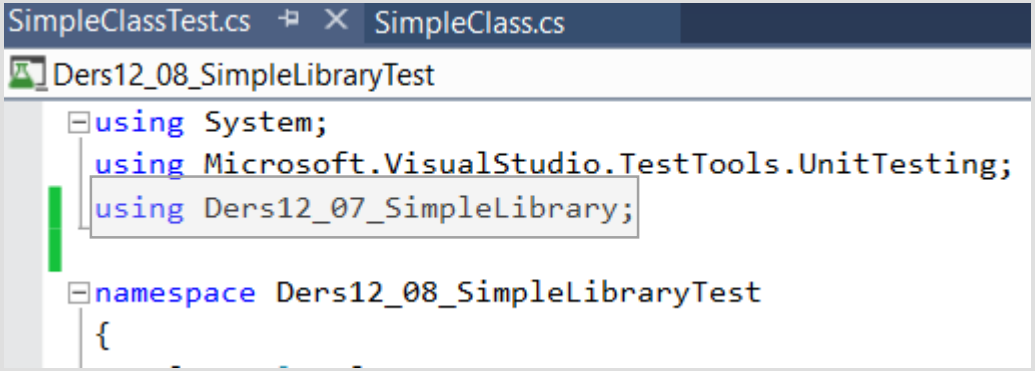
Unit Testing



- Solution Explorer üzerinden projemize yeni bir «Unit Test Project» ekleyelim.

Unit Testing





- References bölümünden test etmek istediğimiz projeyi ekleyelim.
- Projenin NameSpace'ini test sınıfının üst kısmına Using bölümüne ekleyelim ki metotlara rahatça ulaşalım.
- Using Bölümünde Microsoft.VisualStudio.TestTools.UnitTesting olmasına dikkat edin. Bu sınıfın metot ve attribute'lerini kullanacağız.

Unit Testing

```
namespace Ders12_08_SimpleLibraryTest
{
    [TestClass]
    public class SimpleClassTest
    {
        //Test metotu olduğu Atttribute ile bildirilir.
        [TestMethod()]
        public void combineArrayStringWithSpaceTest()
        {
            SimpleClass target = new SimpleClass(); //Test edilecek sınıf tanımlanıyor
            string[] stringArray = null; //null bir array
            string expected = ""; //Beklenen boş vir değer
            string actual;
            //StringArray null olarak fonksiyona gönderilir
            //ne yazık ki try/catch olmadığı için foreach bölümü hatalıdır.
            actual = target.combineArrayStringWithSpace(stringArray);
            Assert.AreEqual(expected, actual);
            Assert.Inconclusive("Bu testi doğrulamak için kodu kontrol edin.");
        }
```

- Sınıfın Test sınıfı olduğunu [TestClass] attribute ile yapıyoruz. (Microsoft.VisualStudio.TestTools.UnitTesting)
- Test metodumuzu yazarken [TestMethod] veya [TestMethod()] attribue'ini kullanıyoruz.
- İki test metot fonksiyonu yazacağız. Birincisi yanlış ikincisi doğru test yapan metotlar olacak. Unutmayın test verinize göre test edilen metot bazen çalışmayabilir.

Unit Testing

```
[TestMethod()]
public void combineArrayStringWithSpaceTest()
{
    SimpleClass target = new SimpleClass(); //Test edilecek sınıf tanımlanıyor
    string[] stringArray = null; //null bir array
    string expected = ""; //Beklenen boş vir değer
    string actual;
    //StringArray null olarak fonksiyona gönderilir
    //ne yazık ki try/catch olmadığı için foreach bölümü hatalıdır.
    actual = target.combineArrayStringWithSpace(stringArray);
    Assert.AreEqual(expected, actual);
    Assert.Inconclusive("Bu testi doğrulamak için kodu kontrol edin.");
}
```

Sonuç alınıyor…

- Assert sınıfı ile karşılaştırma işlemleri yapabilir.
  - AreEqual metodu ile iki verinin karşılaştırması yapılır.
  - Inconclusive ile test onaylanma işleminden sonra sonucun oluşmadığı durumlarda ortaya çıkar. Neticesiz bir işlem yaptığınızı anlarsınız.
  - Assert sınıfındaki diğer metodları internette bulabilirsiniz.
  - Bu metot hatalı çalışacaktır.

Unit Testing

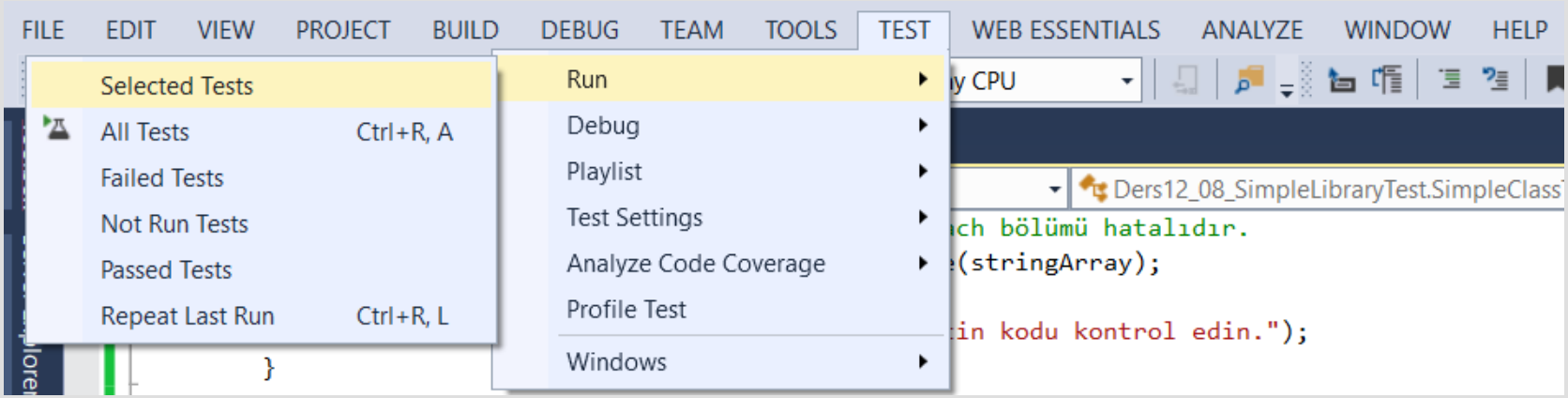
```
//İkinci testimizde aynı metoda başarılı bir test
[TestMethod]
public void PossitiveSenaryoForChecking_combineArrayStringWithSpace()
{
    string expectedResult = "Today is the wonderful day of my life";
    string[] actualStringArray = new string[] { "Today", "is", "the", "wonderful", "day", "of", "my", "life" };
    SimpleClass appObject = new SimpleClass();

    string actualResult = appObject.combineArrayStringWithSpace(actualStringArray);
    Assert.AreEqual(expectedResult, actualResult);
    //Assert.AreEqual<string>(expectedResult, actualResult);//String veri türündeki sonuçlar karşılaştırılıyor.
}
```

- expectedResult ve actualResult eşit üretileceği bir örnek tasarlandı. Test edilen metot'tan istenilen sonuç çıkacaktır.
- Karşılaştırma yaparken string veri geldiğini kesinleştirmek isterseniz:
  - AreEqual<string>(degisken1, degisken2)

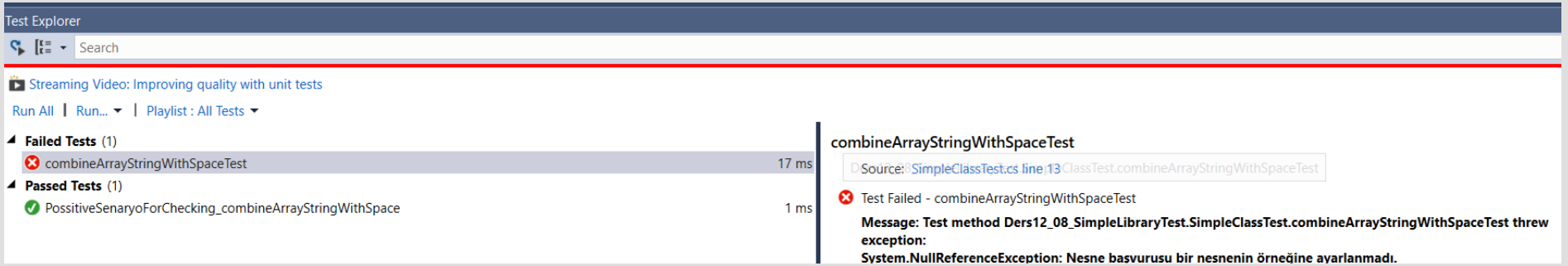
şeklinde bir karşılaştırma yapılır.

Unit Testing



- Son aşama test sonuçlarını yorumlama.
- Uygulama derler gibi TEST bölümünden RUN ve DEBUG yapabiliyoruz. Test işleminde de BreakPoint kullanabiliyoruz.
- Window – Text Explorer bölümünden test sonuçlarını görebiliyoruz.

Unit Testing



- Birinci testimiz başarısız olurken ikinci testimiz başarılı olmuştur.
- Birinci test sonucundaki başarısızlık kodda düzenleme yapılıp düzeltilebilir.
- Fakat, kod yazıldıktan sonra tekrar test edilmelidir. Test işleminin hazırlanması bir zaman kaybı gibi görülse de büyük projelerde metotlarının iyileştirilmesi sonucu yapılabilecek basit hataları minimize eder.
- Test işleminde aynı metoda çok farklı testler uygulayabilirsiniz.
- Test işlemi yapsanız bile beklemedik durumları göz önüne almazsanız kodunuz yine çalışmayacaktır. Eğer böyle bir bug bildirimi alırsanız hem kodunuzu düzenleyin hem de bu bug için yeni bir test metodu yazın.

Design Patterns (Tasarım Desenleri)

- Tasarım desenleri, yazılım tasarımında sürekli karşılaşılan genel sorunlara esnek, yeniden kullanılabilir, başarılı çözümler getiren bir takım hazır kalıplardır.
- Hazır olarak kodun içine konulup çalışabilen, bitmiş tasarımlar değildir. Çeşitli durumlarda sorunların nasıl giderileceğini açıklayan, bunlara yol gösteren açıklamalardır.
- Nesneye dayalı programlamada, tasarım desenleri sınıf ve nesneler arasındaki ilişkilerin en iyi şekilde nasıl olmaları gerektiğini açıklayan yöntemlerdir.
- Algoritmalar, tasarım deseni değildir. Çünkü bunlar hesaplama sorunlarına çözüm getirirler, oysa ki tasarım desenleri yazılım tasarımı sorunlarıyla ilgilenir.

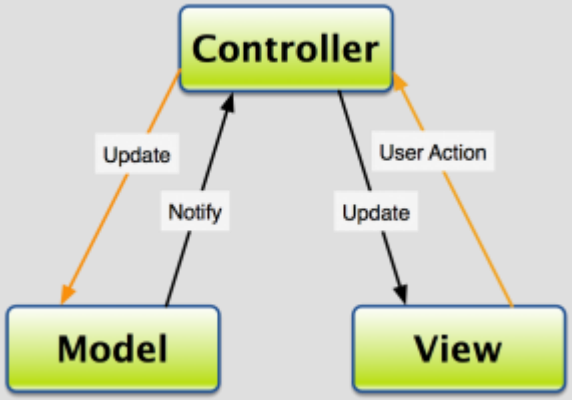
Design Patterns

- Tasarım desenlerini uygulayarak, etkin kodlar üretiriz, yazılımın kalitesini arttırırız. Zamandan ve yazılım için harcanan iş gücünden de tasarruf etmiş oluruz.
- Tasarım Desenleri Üçe Ayrılır.
  - Yaratıcı Tasarım Desenleri
  - Yapısal Tasarım Desenleri
  - Davranışsal Tasarım Desenleri

Bu konu başlı başına bir derstir. İnternette birçok kaynak bulabilirsiniz. Java kodları şeklinde yazılmış ve çok farklı desen tasarımlarını anlatan http://www.tasarimdesenleri.com/jsp/tasdes/tasdesnedir.jsp Adresini öneririm.

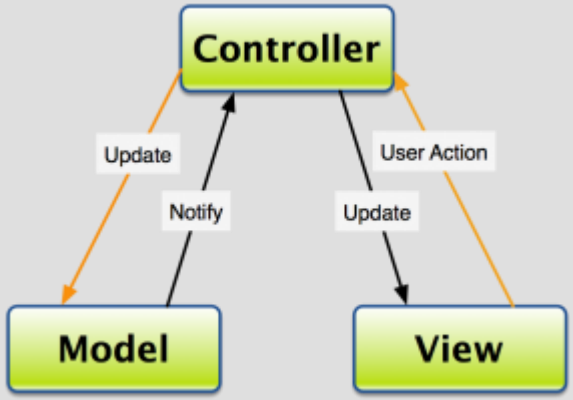
MVC

- MVC (Model-View-Controller), yazdığımız uygulamanın iş mantığı ile (business logic) kullanıcı arayüzünü birbirinden ayrıştıran, uygulamanın farklı amaçlara hizmet eden kısımlarının birbirine girmesini engelleyen yazılım mimarisidir.
- Kodun farklı amaçlara hizmet eden yapılarını birbirinden ayırarak, kodu daha rahat geliştirilebilir ve test edilebilir (yani daha az hata çıkartma potansiyeline sahip) hale getirmiş oluruz.
- C#, Java ve PHP üzerine MVC tabanlı kod ortamları geliştirilmektedir.



MVC

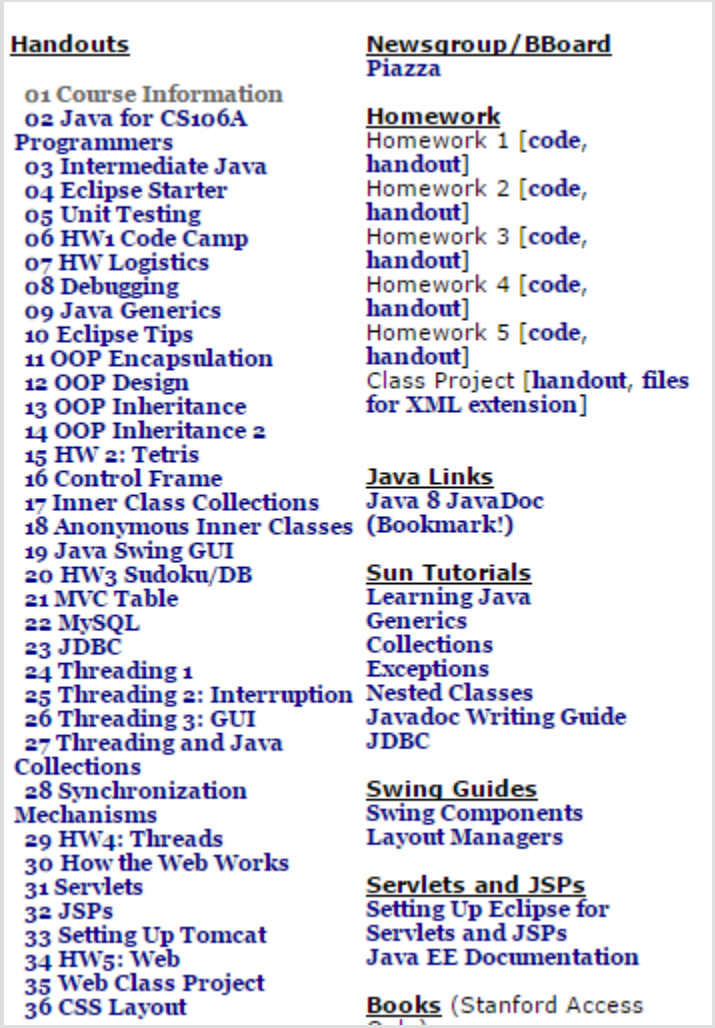
- Model:
  - Uygulamada kullanılan verileri temsil eder ve verilerin işlenme mantığının saklandığı kısımdır.
- View
  - Basitçe, uygulamanızın kullanıcılarınızın gözüyle gördüğü kısmıdır, arayüzdür. (Web, Windows, Tablet, Telefon)
- Controller
  - Model ve View arasında getir götür işlemlerini gerçekleştirir.
  - Kullanıcıların View üzerinden gerçekleştirdiği işlemlerle alınan veriyi Model'e taşır, Model'den aldığı veriyi View üzerinden kullanıcıya gösterir.
  - MVC yapısında ana mantık Model ve View yapısının ayrılmasıdır. Bu iki yapı arasındaki haberleşmeyi sağlayan köprüye Controller diyoruz.



Gelecek Dersler

- Veri Yapıları
- Veritabanı Yönetim Sistemleri
- Yazılım Mühendisliği
- Sistem Analizi ve Tasarımı
- Çevik Yazılım Geliştirme

CS 108 - Object Oriented System Design
Stanford University



Handouts

01 Course Information
02 Java for CS106A Programmers
03 Intermediate Java
04 Eclipse Starter
05 Unit Testing
06 HW1 Code Camp
07 HW Logistics
08 Debugging
09 Java Generics
10 Eclipse Tips
11 OOP Encapsulation
12 OOP Design
13 OOP Inheritance
14 OOP Inheritance 2
15 HW 2: Tetris
16 Control Frame
17 Inner Class Collections
18 Anonymous Inner Classes
19 Java Swing GUI
20 HW3 Sudoku/DB
21 MVC Table
22 MySQL
23 JDBC
24 Threading 1
25 Threading 2: Interruption
26 Threading 3: GUI
27 Threading and Java Collections
28 Synchronization Mechanisms
29 HW4: Threads
30 How the Web Works
31 Servlets
32 JSPs
33 Setting Up Tomcat
34 HW5: Web
35 Web Class Project
36 CSS Layout

Newsgroup/BBoard
Piazza

Homework
Homework 1 [code, handout]
Homework 2 [code, handout]
Homework 3 [code, handout]
Homework 4 [code, handout]
Homework 5 [code, handout]
Class Project [handout, files for XML extension]

Java Links
Java 8 JavaDoc (Bookmark!)

Sun Tutorials
Learning Java
Generics
Collections
Exceptions
Nested Classes
Javadoc Writing Guide
JDBC

Swing Guides
Swing Components
Layout Managers

Servlets and JSPs
Setting Up Eclipse for Servlets and JSPs
Java EE Documentation

Books (Stanford Access

http://web.stanford.edu/class/cs108/

Object-Oriented Programming - MIT

- Interpreter (versus Compiler)
- Expression
- Variable
- Structured Data
- Mutation
- Aliasing
- Procedures and Procedure Calls
- Non-Local Reference
- Environments
- Functions as First-Class Objects
- Object Oriented Programming
- Class
- Instance
- Method
- Inheritance
- Recursion

Practice

- Python
- Datatypes
  - Numbers: ints, bools, float
  - Strings: chars, strings
  - Sequences: strings, lists, tuples
  - Dictionaries: dictionaries
- Multi-Assignment
- Error Messages
- List Comprehensions
- Lambda

http://ocw.mit.edu/courses/electrical-engineering-and-computer-science/6-01sc-introduction-to-electrical-engineering-and-computer-science-i-spring-2011/unit-1-software-engineering/object-oriented-programming/

CEng 546/2 Object Oriented Programming Languages and Systems - ODTÜ

1. Introduction and Overview of C++, Object-Oriented Paradigm, Abstraction, Encapsulation and Type Systems

2. Basic Object-Oriented Concepts and Overloading in C++
(Assignment 1 - Part 1)

3. Inheritance in C++
(Assignment 1 - Part 2)

4. Templates and Exception in C++
(Assignment 1 - Part 3)

5. Midterm Exam – I
(Assignment 1 - Due)

6. Java vs. C++, Introduction to Java classes

7. Information Hiding - Interfaces, Inner classes, Anonymous classes, Abstract classes

8. Error and Exception-Handling
(Assignment 2)

9. Concurrency and Multi-Tasking - Multi-threading, Monitors, Synchronization

10. Midterm Exam – II
(Assignment 2 - Due)

11. Graphical User-Interfaces - Event-handling, MVC architecture

12. Input/Output

13. Interacting with Databases - JDBC

14. Persistency and Object-Serialization, Dynamic Type Identification - RTTI, Reflection
(Assignment 3)

15. Final Exam
(Assignment 3 - Due)

http://www.ceng.metu.edu.tr/secondprog/ceng546/

BLG 252E - Object-Oriented Programming - İTÜ

Dersin Haftalık Planı

Hafta	Konu
1	INTRODUCTION Why Object Technology?
2	C++ : As a Better C Non-object-oriented features
3	Classes and Objects Controlling Access to Members
4	Initializing Class Objects: CONSTRUCTORS DESTRUCTORS
5	Constant Objects and Const Member Functions Static Class Members NESTING OBJECTS: Classes as Members of Other Classes
6	1st Midterm
7	Operator Overloading
8	INHERITANCE Redefining Members (Name Hiding) Access Control
9	Special Member Functions and Inheritance Multiple Inheritance POINTERS TO OBJECTS
10	POLYMORPHISM
11	2nd Midterm
12	The Unified Modeling Language - UML EXCEPTIONS
13	Templates
14	THE STANDARD TEMPLATE LIBRARY (STL)

http://ninova.itu.edu.tr/tr/dersler/bilgisayar-bilisim-fakultesi/21/blg-252e/haftalikplan

csharp-station

[html]	[code]	Lesson 01: Getting Started
[html]	[code]	Lesson 02: Expressions, Types, and Variables
[html]	[code]	Lesson 03: Control Statements - Selection
[html]	[code]	Lesson 04: Control Statements - Loops
[html]	[code]	Lesson 05: Methods
[html]	[code]	Lesson 06: Namespaces
[html]	[code]	Lesson 07: Introduction to Classes
[html]	[code]	Lesson 08: Class Inheritance
[html]	[code]	Lesson 09: Polymorphism
[html]	[code]	Lesson 10: Properties
[html]	[code]	Lesson 11: Indexers
[html]	[code]	Lesson 12: Structs
[html]	[code]	Lesson 13: Interfaces
[html]	[code]	Lesson 14: Introduction to Delegates and Events
[html]	[code]	Lesson 15: Introduction to Exception Handling
[html]	[code]	Lesson 16: Using Attributes
[html]	[code]	Lesson 17: Enums
[html]	[code]	Lesson 18: Overloading Operators
[html]	[code]	Lesson 19: Encapsulation
[html]	[code]	Lesson 20: Introduction to Generic Collections
[html]	[code]	Lesson 21: Anonymous Methods
[html]	[code]	Lesson 22: Topics on C# Type
[html]	[code]	Lesson 23: Working with Nullable Types

http://www.csharp-station.com/Tutorial.aspx

Diğer bir site:

- http://www.tutorialspoint.com/csharp/csharp_overview.htm

Kaynaklar

- http://www.tutorialspoint.com/csharp/csharp_overview.htm
- http://ileriprogramlama.blogspot.com.tr/2012/09/multithread-programlamann-temelleri.html
- http://www.csharpnedir.com/articles/read/?id=5
- http://www.codeproject.com/Articles/391465/Creating-Unit-tests-for-your-csharp-code
- http://orhanoztekin.blogspot.com.tr/2014/06/unit-test-yaparken-skca-karslaslan.html
- http://www.tasarimdesenleri.com/jsp/tasdes/tasdesnedir.jsp
- http://koddit.com/yazilim/mvc-nedir-gercek-orneklerle-mvc-nedir-anlayalim/